Mini Payment Demo 개인 미니 프로젝트

2026.02

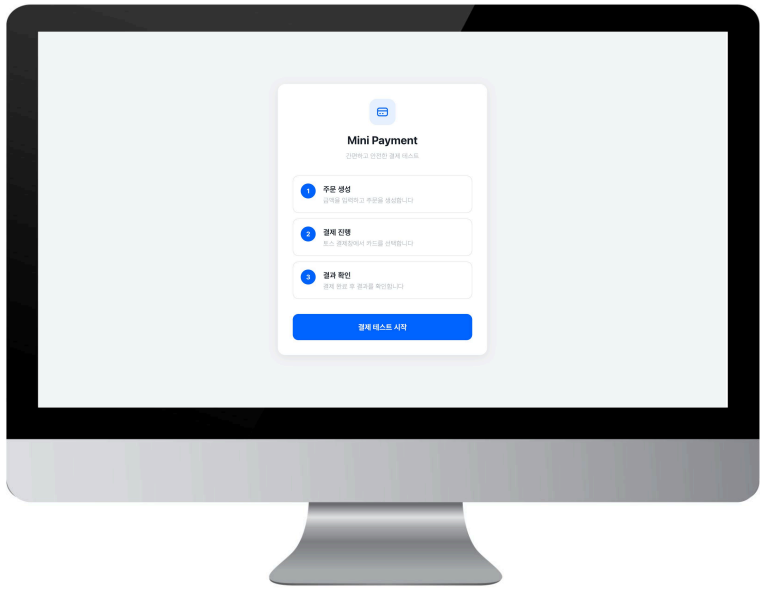
프로젝트 개요

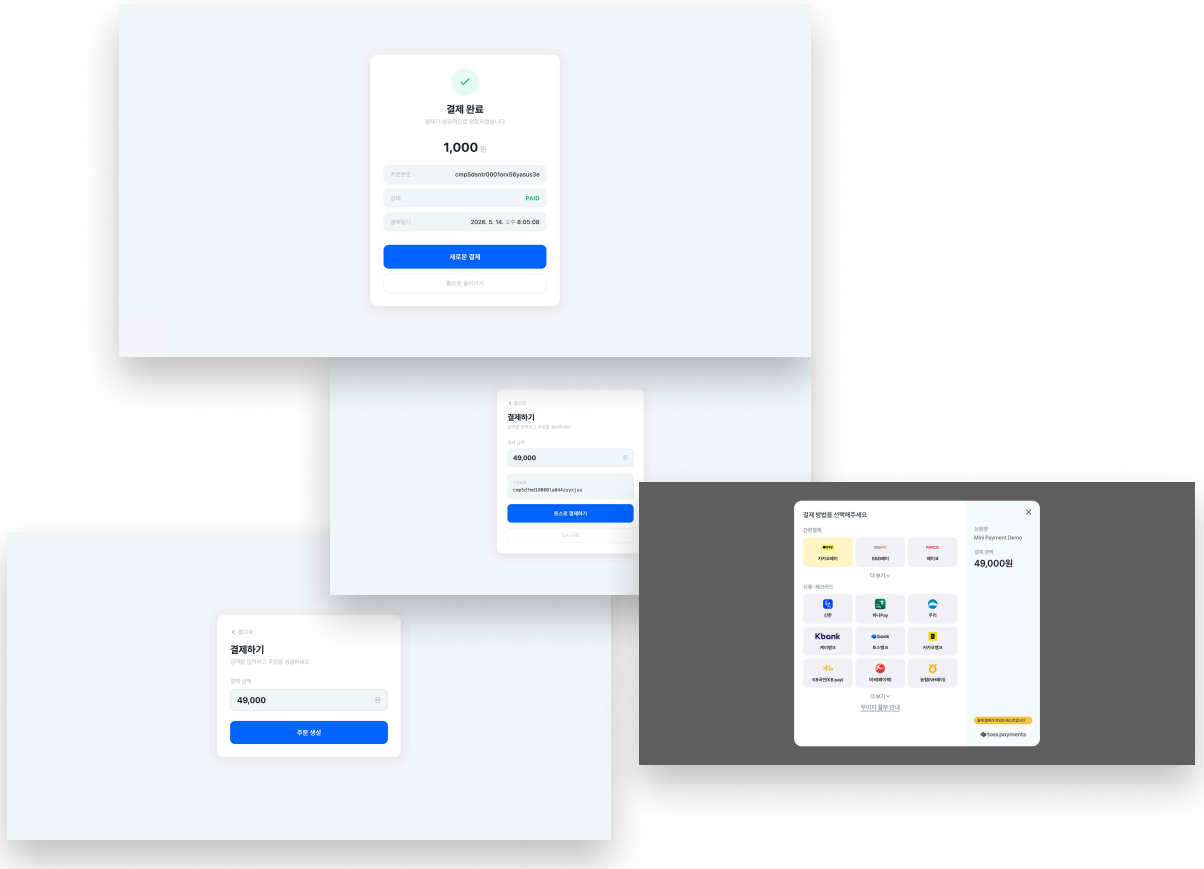
주문 생성 → 결제 요청 → 승인 검증 → 결제 상태 저장까지 구현한 TossPayments 기반 결제 플로우 미니 프로젝트

목표

서버 검증 결과와 DB 주문 상태를 기준으로 결제 결과를 판단

결제 성공·대기 상태를 사용자에게 명확히 표시하는 UI를 구현





Mini Payment Demo 개인 미니 프로젝트

2026.02

기술 스택

Front: Next.js, TypeScript, Tailwind CSS

Back-end: Next.js Route Handler, Prisma, PostgreSQL

PG: TossPayments 테스트 환경

Deploy: Vercel

역할

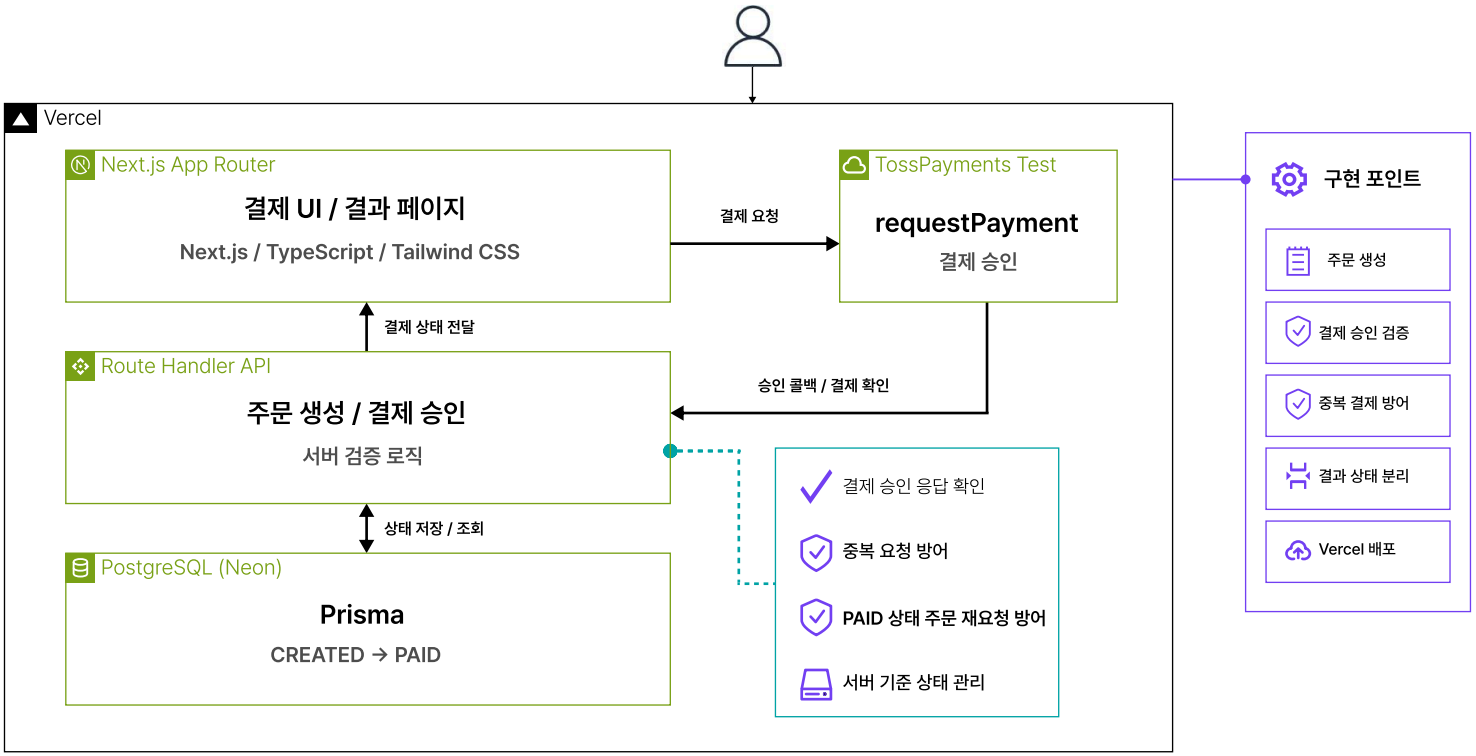
Next.js → 주문 생성부터 결제 결과 확인까지의 사용자 플로우 구현

Route Handler → 결제 승인 API 및 서버 기준 상태 검증 로직 구현

Prisma / PostgreSQL → 주문 상태(CREATED → PAID) 관리

TossPayments → requestPayment 기반 테스트 결제 연동

중복 방어 → 이미 결제된 주문에 대해 기존 결과를 반환하는 로직 구현



Mini Payment Demo 개인 미니 프로젝트

2026.02

결제 상태 신뢰성

문제

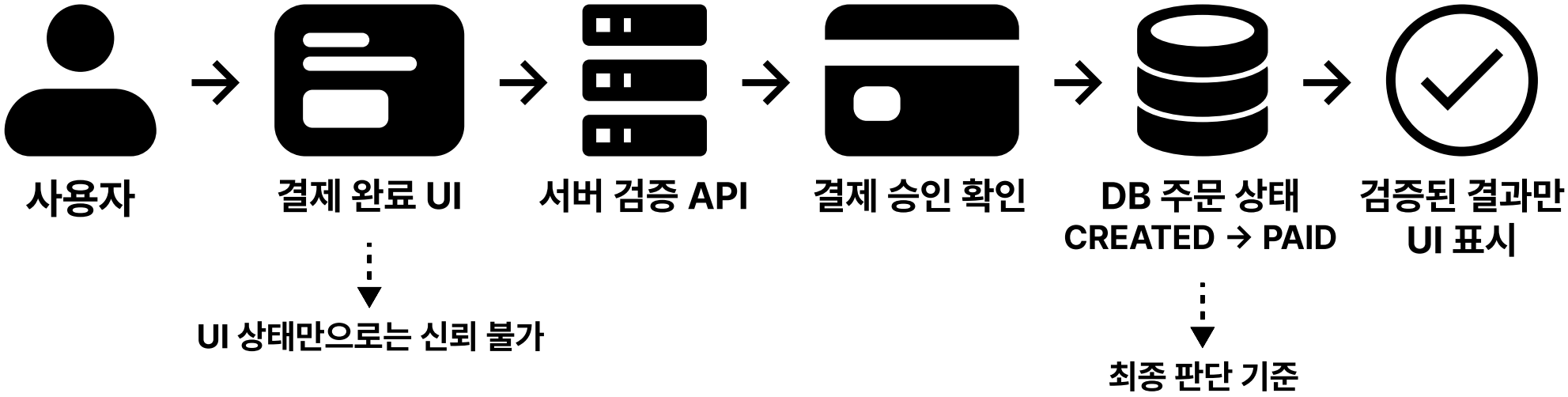
클라이언트의 결제 완료 상태만으로는 실제 결제 성공을 보장할 수 없음

고민

결제의 최종 판단 기준을 UI가 아닌 서버와 DB 상태로 두어야 함

해결

DB 주문 상태를 CREATED → PAID로 관리하고, 서버 승인 검증 결과를 기준으로 결과 페이지의 UI 상태를 표시하도록 구현



Mini Payment Demo 개인 미니 프로젝트

2026.02

중복 요청 방어

문제

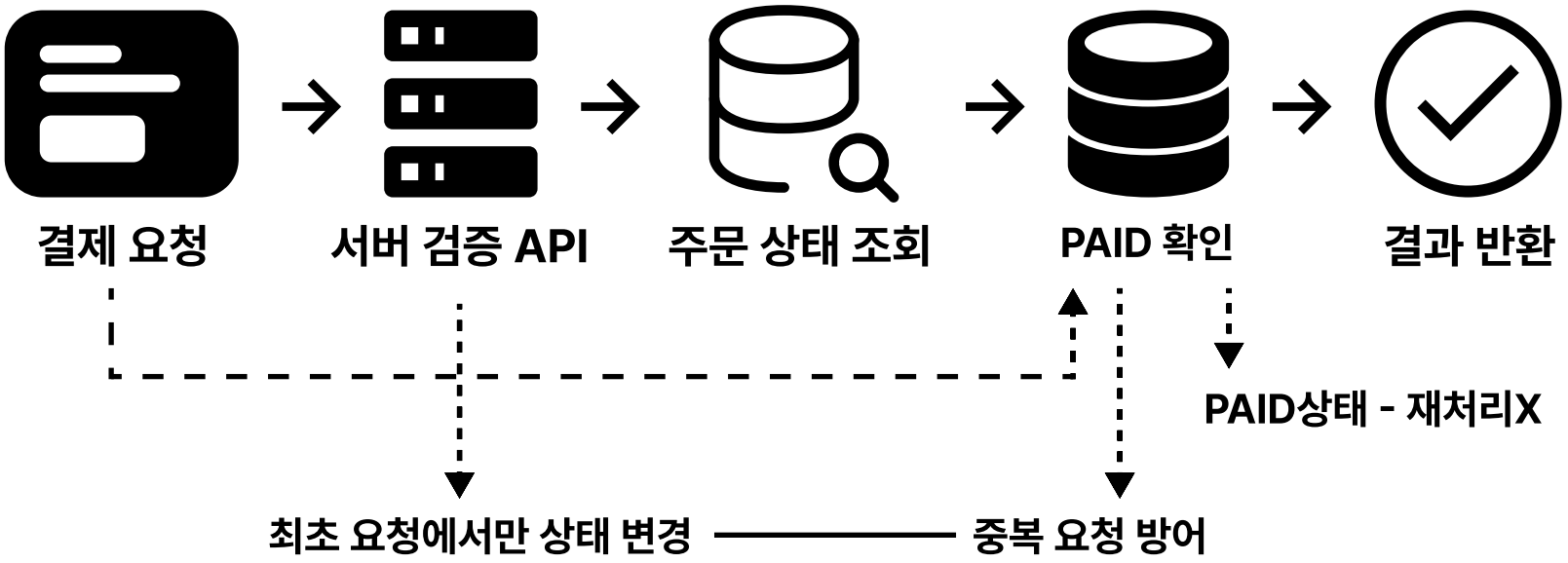
동일한 orderId로 결제 승인 요청이 반복될 경우, 이미 처리된 주문에 대해 상태 변경이 다시 시도될 수 있음

고민

재시도 상황에서도 주문 상태는 한 번만 PAID로 변경되고, 이후 요청은 기존 결과를 반환해야 함

해결

주문 상태가 PAID인 경우 추가 결제 처리를 제한하고, 기존 결제 결과를 반환하도록 조건 분기 처리



Mini Payment Demo 개인 미니 프로젝트

2026.02

결제 결과 상태 구분

문제

결제 요청 이후 사용자가 현재 주문이 결제 완료 상태인지, 아직 대기 상태인지 결과 페이지에서 명확히 판단하기 어려움

고민

클라이언트 화면 상태가 아니라 DB에 저장된 주문 상태를 기준으로 결제 결과를 표시해야 함

해결

DB 주문 상태를 기준으로 결제 성공(PAID)과 대기(CREATED) 상태를 구분하고, 결과 페이지에서 아이콘과 상태 텍스트로 결제 진행 상태를 표시



PAID

정상 결제 완료



CREATED

결제 대기

Mini Payment Demo 개인 미니 프로젝트

2026.02

프로젝트 회고

배운 점

결제 결과 UI는 클라이언트의 성공 화면이 아니라, 서버 검증 결과와 DB 주문 상태를 기준으로 표시해야 한다는 점을 배웠습니다.

아쉬운 점

현재는 PAID 상태 여부를 기준으로 기본적인 재요청 방어를 처리했지만, 실패·취소 상태와 Webhook 예외 상황에 대한 처리는 충분하지 않았습니다.

개선방향

결제 승인 응답 검증과 실패·취소 상태 처리를 보강하고, Webhook 이벤트 수신 상황에서도 주문 상태가 일관되게 유지되도록 개선하고자 합니다.